포트폴리오 설명서

- **이름:** 김두영
- **지원 직무:** 백엔드

0. 개요

> 본 문서는 함께 제출한 `portfolio.pdf`의 내용을 보충하는 설명서입니다. 주요 프로젝트를 **소개**, **핵심 구현과 설계 판단**, **아쉬운 점과 개선 방향**으로 나눠 설명했습니다.

1. Real-time Messaging App

1.1 프로젝트 소개

로그인한 두 사용자가 실시간으로 메시지를 주고받을 수 있는 서비스입니다. JWT를 이용한 인증 시스템과 WebSocket 통신을 학습하는 것을 목표로 진행했습니다.

1.2 핵심 구현과 설계 판단

1.2.1 JWT 인증

`HttpOnly` 쿠키 기반 JWT를 구현했습니다. AT(Access Token)와 RT(Refresh Token)로 로그인 상태를 관리했습니다.

쿠키 옵션

프론트엔드와 백엔드가 배포 환경에서 서로 다른 사이트로 제공되는 점을 고려하여, 쿠키 옵션 `sameSite`와 `secure`를 `NODE_ENV`에 따라 다르게 적용하도록 구성했습니다.

`NODE_ENV`	`sameSite`	`secure`
`development`	`lax`	`false`
`production`	`none`	`true`

각 실행 환경에서 AT와 RT의 쿠키 옵션이 의도한 대로 반환되는지 검증하는 단위 테스트를 작성했습니다.

RT 세션 교체

이전 RT의 재사용을 막기 위해 AT 갱신 시 새 RT를 발급하고 이전 RT를 무효화합니다. DB에 저장된 이전 RT 세션을 삭제하고 새 RT 세션을 등록하며, 두 요청을 트랜잭션으로 처리해 RT 세션이 완전히 유실되는 것을 방지했습니다.

중복된 `tokenHash`로 RT 세션 생성을 요청하여 고유 제약 조건 위반을 유도하고, 기존 세션이 그대로 유지되는지 확인했습니다.

1.2.2 WebSocket

로그인한 사용자가 대화방에서 메시지를 보내면, 서버는 그 대화방에 참여하고 있는 다른 사용자(수신자)의 WebSocket에 그 메시지를 전달합니다. 같은 사용자가 여러 기기에서 접속한 경우 동시에 그 메시지를 수신합니다.

WebSocket Registry

각 사용자의 WebSocket 연결은 `Map<number, Set<WebSocket>>` 인터페이스를 구현한 Registry에서 관리합니다. WebSocket 서버에 연결할 때 AT에서 사용자의 아이디를 추출하고, 해당 아이디에 대응하는 WebSocket `Set`을 새로 생성하거나 이미 존재하는 `Set`에 새 WebSocket을 추가하여 사용자와 WebSocket 연결을 매핑합니다.

Message Publisher

메시지 이벤트의 생성과 WebSocket으로의 전달은 Publisher가 담당합니다. Publisher는 Registry를 참조하며 수신자 아이디, 대화 아이디와 메시지를 인자로 받는 콜백 함수로 호출됩니다. Registry에서 수신자 아이디에 대응하는 WebSocket `Set`을 꺼내어 순회하고, 연결된 WebSocket에 메시지 생성 이벤트를 전달합니다.

설계 초기에는 Registry를 WebSocket 서버가 생성했기 때문에, Controller는 Registry에 직접 접근하지 못하고 WebSocket 서버를 HTTP 요청을 통해 역추적해서 이벤트 전달 함수를 호출했습니다. Registry 생성 주기를 WebSocket 서버에서 분리하고, 이벤트 전달 함수를 Publisher로 추출하여 WebSocket 서버와 동일한 Registry를 공유하게 함으로써 구현 맥락을 명료화했습니다.

1.3 아쉬운 점과 개선 방향

1.3.1 대화 나가기 기능

화면 구현 후 실제로 브라우저에서 테스트해 본 결과, 필요 없는 대화방을 정리할 수 있는 기능이 필요하다고 생각했습니다. 사용자가 대화에서 나가면 목록에서 해당 대화를 삭제하고, 상대방이 다시 메시지를 보내면 목록과 대화를 복구하되 대화 기록은 해당 메시지부터만 조회할 수 있게 하려고 합니다. 상대방의 목록과 대화 기록은 영향을 받지 않습니다.

1.3.2 로그인 유지 정책

현재 구현에서 RT는 AT를 갱신할 때마다 함께 새로 발급하고, 최대 로그인 기한은 정해져 있지 않습니다. 이 정책은 무효화된 RT의 재사용 위험은 방지하지만, 공격자가 유효한 RT를 탈취하여 계속 AT를 갱신하는 상황은 방지하지 못합니다. AT 갱신이 계속될 수 있는 기간을 제한하기 위해 최대 로그인 기한을 도입하려 합니다.

2. MatchingBot

2.1 프로젝트 소개

구직 회원과 기업 회원이 이용하는 HR 매칭 서비스입니다. 이력서와 채용 공고에서 키워드를 추출하고 둘 사이의 유사도를 분석하여 사용자에게 제공했습니다. 구직자와 기업은 매칭률 내림차순으로 각각 채용 공고와 이력서 목록을 조회할 수 있고, 분석 대상을 조회 화면에서 교체할 수 있습니다.

팀장으로 회의 진행과 전체 일정을 조율했고, Git 관리를 맡아 코드 검토와 병합을 담당했습니다. 사용자 인증과 구직·기업·관리자 회원 유형에 따른 인가 시스템을 구현했습니다. 기업 사용자의 보고서 생성 기능, 채용 공고 리뷰 기능과 관리자 페이지 전반을 구현했습니다. CI/CD와 클라우드 구축 및 배포를 맡았습니다.

2.2 핵심 구현과 설계 판단

2.2.1 기업 보고서 생성

기업 회원 신규 가입·정보 변경 시 기업 기본 정보, 재무 정보, 최근 채용 활동 등을 서버로 전달합니다. 서버는 OpenAI API를 통해 기업 소개, 재정, 고용, 성장, 평판에 대한 점수 및 분석이 포함된 보고서를 JSON 형식으로 응답받고, 이를 전체·요약 보고서 이미지로 변환해 S3에 저장합니다.

초기 설계에서는 가입·수정 요청과 보고서 생성 요청을 동기로 처리하여 가입 요청의 응답 대기 시간이 길어지는 문제가 있었습니다. 이를 해결하기 위해 보고서 생성 요청을 비동기로 분리하여 가입·수정 요청이 보고서 생성·업로드 완료를 기다리지 않도록 했습니다.

2.2.2 채용 공고 리뷰

신규 채용 공고 등록 시 공고 내용과 「채용절차의 공정화에 관한 법률」 페이지 링크를 AI에 전달해 해당 페이지를 참고한 수정안을 요청합니다. 응답받은 수정안 JSON을 원본과 비교할 수 있는 화면에 표시합니다. 기업 회원은 수정안을 검토한 후 적용하거나 원본으로 복구할 수 있습니다.

2.2.3 인프라·배포

AWS VPC를 `us-east-2` 리전에 구축했습니다. `2a`, `2c` 두 AZ에 각각 퍼블릭 서브넷과 프라이빗 서브넷을 배치했습니다. `2a`의 퍼블릭 서브넷에 Jenkins와 Docker 기반의 Spring Boot·FastAPI 앱을 실행하는 EC2를 배치했습니다. MySQL RDS는 서브넷 그룹을 통해 프라이빗 서브넷에 배치되며 VPC의 `local` 경로를 통해 각 앱과 통신합니다.

AWS EIP를 Jenkins와 앱이 있는 EC2에 연결했습니다. 사용자는 브라우저를 통해 이 IP로 접근하고, EC2와 Spring Boot 컨테이너 간의 Docker 포트 매핑을 통해 실제 앱에 접근할 수 있습니다.

서버 관리자는 로컬 PC에서 MobaXterm의 SSH 연결을 통해 Jenkins와 각 앱이 포함된 EC2에 접근하여 컨테이너 실행 로그 등을 확인하고 디버깅할 수 있습니다. `2a`의 퍼블릭 서브넷에 Bastion을 배치하고, 로컬 DBeaver에서 Bastion을 경유하는 SSH 터널을 연결해 DB를 직접 관리할 수 있습니다.

`deploy` 브랜치에 푸시하면 Jenkins가 웹훅으로 변화를 감지해 새 Docker 이미지를 빌드한 후, 할당된 ECR 저장소에 푸시하여 이미지 교체·앱 재실행을 진행합니다.

2.3 아쉬운 점과 개선 방향

2.3.1 기업 보고서 DB 도입

변환된 최종 이미지를 업로드하고 그 주소만 DB에 저장하는 현재 구현은 DB 구조가 단순하다는 장점이 있지만, JSON → HTML → PDF로 이어지는 각 과정에서 발생하는 처리 비용은 개선의 여지가 있습니다. 보고서의 내용을 담은 JSON을 DB에 저장하고 필요할 때 화면을 렌더링하면 해당 비용을 절약할 수 있을 것으로 기대합니다.

2.3.2 운영 안정성 개선

현재는 앱 실행과 이미지 빌드를 한 인스턴스에서 담당하기 때문에, 빌드 작업이 서비스 성능을 저하시킬 수 있으며, 인스턴스 재부팅 시 서비스가 중단됩니다. Jenkins 전용 인스턴스와 앱 전용 인스턴스를 분리하면 빌드 환경이 서비스에 미치는 영향을 줄일 수 있을 것입니다.

앱 인스턴스를 각 AZ에 배치하고, ALB가 각 AZ를 사용하도록 구성해 운영 중인 앱 인스턴스들을 참조하게 하면, 어느 한쪽에 장애가 생기더라도 정상인 앱 인스턴스로 트래픽을 유도하여 장애 상황에 대응할 수 있을 것입니다. 또한 요청을 여러 인스턴스로 분산시켜 한 서버에만 부하가 집중되는 상황도 줄일 수 있습니다.

2.3.3 접근·통신 보안 개선

현재는 빌드·앱 공인 IP가 인터넷 전역에 노출되어 있고, 해당 포트로 외부의 접근이 가능해 잠재적 보안 위험이 있습니다. 각 인스턴스를 프라이빗 서브넷으로 이전하면, 앱 인스턴스는 ALB를 통한 요청만 수신하여 앱 서버에 대한 직접 접근을 제한할 수 있습니다. 빌드 인스턴스 또한 Jenkins 관리 화면과 SSH를 Bastion을 통해 보호할 수 있을 것입니다.

현재 서비스는 HTTP 환경으로 배포되고 있어, 서비스에 필요한 브라우저의 `Secure` 쿠키와 사용자의 위치 조회 기능을 사용하지 못합니다. Route 53을 이용해 도메인을 발급받고, ACM을 통해 해당 도메인의 인증서를 발급받아 ALB의 HTTPS 443 리스너에 연결하면 해당 기능들을 설계한 그대로 제공할 수 있을 것입니다.